

DevProof: Evidence-Based Developer Skill Assessment and Interview Support Platform

Semester Project Proposal

Khushi Agarwal
Roll No: 1024170221

Swathi Gurada
Roll No: 1024170106

Rohit Singhal
Roll No: 1024170213

Submitted to: Ms.Sandeep Kaur
Computer Science and Engineering Department (CSED)
Thapar Institute of Engineering and Technology
Course: UCS503— Software Engineering
20.08.2026

Elevator Pitch

Gap: Technical skills listed on resumes are often difficult and time-consuming to verify, because the supporting evidence is scattered across public development platforms rather than presented in a reviewable form.

Solution: DevProof extracts claimed skills from candidate resumes and analyzes publicly available GitHub evidence to produce transparent, explainable skill-support assessments.

Impact: The system aims to reduce the manual effort recruiters spend verifying resume claims and to help interviews focus on the skills that most need further confirmation.

1 Introduction / Problem Statement

Candidate resumes for software engineering roles routinely list technical skills such as Python, React, Docker, or FastAPI. Confirming that these claims are backed by real development experience currently falls on the recruiter, who must manually open GitHub profiles, inspect repositories, check dependency files, and judge whether a technology was genuinely used in a project. This process is repeated for every candidate and every claimed skill, which makes it slow and inconsistent, particularly when a placement team is screening a large applicant pool.

The core gap is not that public evidence is unavailable — most active developers maintain some form of public repository history — but that this evidence is scattered, unstructured, and time-consuming to interpret consistently across reviewers. A recruiter reviewing ten candidates in an afternoon has little time to trace commit histories or distinguish an original project from a forked one, so verification is often skipped or done superficially.

DevProof addresses this gap by automatically collecting and organizing publicly available development evidence relevant to a candidate's claimed skills, and by presenting it through a transparent, rule-based scoring system. The platform does not attempt to determine whether a candidate genuinely knows a technology, nor does it make any claim about honesty; it evaluates and reports the strength of *publicly available evidence supporting a claimed skill*, leaving the final judgment to the human reviewer. Where evidence is insufficient, the system generates a targeted interview question so that verification effort is concentrated where it is most needed.

2 Objectives

Overarching goal: To design, implement, and evaluate a web-based platform that assists recruiters in assessing the evidential support for technical skills claimed on candidate resumes, using publicly available GitHub activity.

1. Develop a deployable web platform that extracts technical skills, projects, and GitHub links from an uploaded resume PDF.
2. Implement a rule-based evidence engine that identifies supporting repository evidence for a defined set of technical skills (Section 3.1).
3. Develop an explainable scoring mechanism that classifies each claimed skill as *Strong Evidence*, *Good Evidence*, or *Needs Verification*, with a stated reason for each classification.
4. Evaluate whether DevProof produces a measurable reduction in manual skill-verification time compared with an unassisted manual review process.
5. Implement automated testing and a CI/CD pipeline to support short weekly development iterations and repeatable staging deployment.

3 Methodology

The project follows an engineering methodology: system design, incremental implementation, automated testing, and measured evaluation, rather than algorithmic research.

3.1 System Architecture

DevProof follows a standard three-tier web architecture (Figure 1). The React/Tailwind frontend submits resumes and displays evidence dashboards; the FastAPI backend performs parsing, GitHub retrieval, fingerprinting, and scoring; and a PostgreSQL database persists candidates, skills, and evidence records. The backend additionally calls the public GitHub REST API for repository metadata.

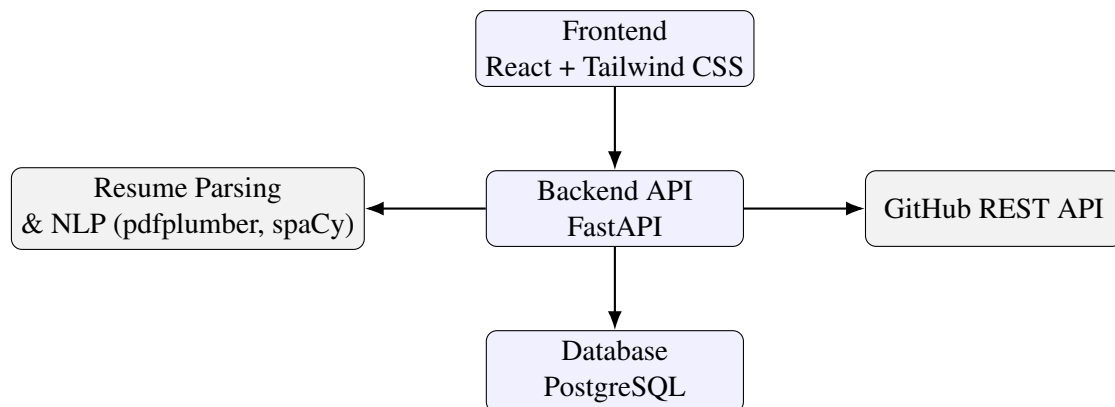


Figure 1: DevProof system architecture: three-tier web application with external GitHub and NLP integration.

3.2 Resume Processing and Skill Extraction

Uploaded resumes are parsed with `pdfplumber` to extract raw text, from which candidate name, project descriptions, and GitHub links are identified. Skill names are extracted and normalized (e.g.,

“ReactJS” and “React.js” both map to “React”) using a curated mapping combined with `spaCy`-based entity recognition. Optional semantic matching via `sentence-transformers` may be used to associate related phrasings (e.g., “built REST services” with “developed REST APIs”); no custom model is trained.

3.3 GitHub Evidence Engine and Technology Fingerprinting

For each candidate’s public repositories, the backend retrieves language composition, dependency manifests, configuration files, commit activity, and fork status via the GitHub REST API. A technology fingerprint set defines, per technology, the concrete signals that constitute evidence — for example, a `Dockerfile` or `docker-compose.yml` for Docker, or a `react/react-dom` dependency together with JSX/TSX files for React. The initial supported set is intentionally limited to Python, Java, C++, JavaScript, React, Node.js, Express, Flask, FastAPI, Django, MongoDB, MySQL, PostgreSQL, and Docker.

3.4 Evidence Scoring and Explainability

Each detected signal contributes a configurable weight toward a 0–100 evidence score (Table 1); thresholds map the score to *Strong Evidence*, *Good Evidence*, or *Needs Verification*. The initial weights are heuristic starting points to be reviewed and tuned during the project, not a claim of statistical optimality. Every score is accompanied by the specific evidence items that produced it, so the classification is always explainable rather than a black-box output.

Table 1: Illustrative rule-based scoring for a React claim (weights are configurable).

Evidence Signal	Weight
React dependency detected in <code>package.json</code>	+30
JSX/TSX files present	+20
Original repository (not a fork)	+15
Meaningful commit activity	+15
Recent development activity	+10
Example total	90 / 100 – Strong Evidence

3.5 Interview Verification Support

When evidence for a claimed skill is insufficient, DevProof reports *Needs Verification* with a stated reason (e.g., “no Docker-specific configuration found in analyzed repositories”) and generates a targeted, template-based interview question the recruiter can use, such as “How did you use Docker to containerize this application?” An LLM may optionally be used to phrase these questions but is not a core dependency.

3.6 Database, Testing, and Deployment

The relational schema centers on `Candidate`, `Skill`, `CandidateSkill`, `Repository`, `Evidence`, `Project`, and `InterviewQuestion` entities, kept intentionally simple for a semester scope. Backend logic is tested with `pytest` and linted with `Ruff`; the frontend is tested with `Vitest` and `React Testing Library` and linted with `ESLint`. Every pull request triggers the pipeline in Figure 2; on a successful merge, the application is deployed to staging (and later production) using free-tier hosting appropriate to each layer.

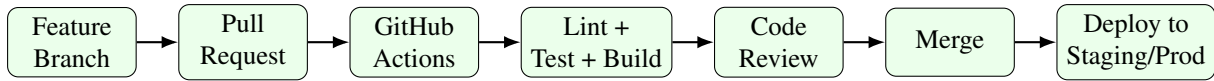


Figure 2: CI/CD workflow: every change is linted, tested, and reviewed before merge and deployment, keeping releases small and safe.

Short weekly iterations paired with this pipeline give fast feedback, reduce the risk of broken code reaching the main branch, and keep every increment independently deployable and demonstrable.

4 Timeline

The project is planned over a 13-week semester using six short iterations plus a buffer week, following the time-to-value approach of delivering a working increment early and refining it (Table 2).

Table 2: Iteration-wise development timeline.

Week(s)	Iteration	Deliverable
1–2	Resume Intelligence	Architecture, project setup, PDF upload, text/skill extraction, GitHub URL extraction, basic CI
3–4	GitHub Evidence Engine	GitHub API integration, repository retrieval, language and fork detection, activity analysis
5–6	Technology Fingerprint Engine	Dependency and config-file detection rules for the initial technology set
7–8	Evidence Scoring	Transparent scoring engine, Strong/Good/Needs Verification classification with explanations
9–10	Dashboard + Interview Support	Candidate dashboard, skill detail views, interview question generation
11–12	Testing, Evaluation & Refinement	Pilot testing, metric collection, scalability improvements, final staging deployment
13	Buffer	Contingency for delays, documentation, final review

5 Evaluation

Since the platform has not yet been built, no results are claimed; the table below states the metrics and planned evaluation method (Table 3), using a pilot of approximately 10–20 candidate resume/GitHub examples and 3–5 human reviewers, where feasible.

Table 3: Planned evaluation metrics.

Metric	Planned Measurement
Verification time reduction (primary)	Compare time for a human reviewer to manually verify skills vs. using DevProof
Human–system agreement	Compare DevProof classifications with independent human reviewer assessments
Evidence coverage	% of claimed skills for which supporting public evidence is found
User clarity	Reviewer rating (1–5 scale) of how clearly a classification is explained
Reliability	Successful analysis requests vs. API/system failures during pilot use
Interview-question usefulness	Reviewer rating of relevance and usefulness of generated questions

6 Scalability

The frontend/backend separation, a stateless FastAPI layer, and pagination/indexing in the database provide a practical path from a handful of pilot candidates to a larger volume, without any claim that large-scale performance has been demonstrated. Caching of GitHub API responses and persisting previously computed evidence reduce redundant external calls, and computationally heavier analysis can be moved to background jobs if pilot testing shows the need.

7 Limitations

- Public GitHub evidence supports but does not prove actual developer skill.
- Private repositories cannot be analyzed without explicit candidate access.
- An original (non-fork) repository does not by itself prove sole authorship, and team projects make individual attribution difficult.
- Resume parsing may fail for unusual PDF layouts, and technology detection can produce false positives or negatives.
- The GitHub API is subject to rate limits, which constrain batch analysis volume.
- Evidence scores are decision-support indicators for recruiters, not objective measurements of ability.

8 Risks and Mitigations

Table 4: Identified risks and corresponding mitigations.

Risk	Mitigation
GitHub API rate limits	Cache responses; minimize repeated requests per candidate
Resume parsing failures	Allow manual correction of extracted GitHub URL/skills
Incomplete public evidence	Report “Needs Verification” rather than a negative judgment
Incorrect or noisy scoring	Transparent, adjustable rules combined with human evaluation
Privacy concerns	Data minimization and appropriate access/deletion controls
Scope expansion	Prioritize the defined MVP; defer optional features to later iterations
Deployment failures	Staging environment and CI/CD before any production deploy
Team integration conflicts	Feature branches, pull requests, and mandatory automated CI checks

9 Resources / Budget

Human resources: A three-member student team with defined roles across backend/evidence-engine development, frontend/dashboard development, and testing/documentation.

Software resources: Python, FastAPI, React, PostgreSQL, Git/GitHub, GitHub Actions, and standard open-source Python/JavaScript libraries (pdfplumber, spaCy, pytest, ESLint, Ruff).

Hardware: Standard development laptops and an internet connection; no specialized lab equipment is required.

External services: The public GitHub REST API and a suitable free/educational hosting tier for staging deployment. The prototype requires no external funding, since all core tools are open-source; however, deployment costs are not guaranteed to remain free indefinitely and depend on the hosting provider’s current free-tier policy at deployment time.